



Week3 Lab Thread - bài tập tuần 3 môn lập trình phân tán java

Lập trình phân tán với công nghệ Java (Trường Đại học Công nghiệp Thành phố Hồ Chí Minh)



Scan to open on Studeersnel

Week 3 – Lab Fork/Join Framework

Exercise 1

Write a program recursively finds the sum of an array in parallel using Java's Fork/Join Framework:

1. RecursiveTask
2. RecursiveAction

Exercise 2

Write a program Merge sort - using Java's Fork/Join Framework.

Exercise 3

Write a program - using Stream parallel:

```
public class Student {
    private String firstName;
    private String lastName;
    private double age;
    private int grade;
    private boolean isCurrent;
    public Student(String firstName, String lastName, double age, int grade,
boolean isCurrent) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.age = age;
        this.grade = grade;
        this.isCurrent = isCurrent;
    }
    public String getFirstName() {
        return firstName;
    }
    public String getLastName() {
        return lastName;
    }
    public double getAge() {
        return age;
    }
    public int getGrade() {
        return grade;
    }
    public boolean checkIsCurrent() {
        return isCurrent;
    }
}

private static String[] firstNames = {"Thanh", "Thao", "Tuan", "Hong", "Lan",
"Phuong"};
private static String[] lastNames = {"Nguyen", "Le", "Tran", "Mai", "Lam",
"Ha"};
private static Student[] generateStudentData() {
    final int N_STUDENTS = 80_000_000;
    final int N_CURRENT_STUDENTS = 1_000_000;
    Student[] students = new Student[N_STUDENTS];
    Random r = new Random(123);
    for (int i = 0; i < N_STUDENTS; i++) {
        final String firstName = firstNames[r.nextInt(firstNames.length)];
        final String lastName = lastNames[r.nextInt(lastNames.length)];
        final double age = r.nextDouble() * 100.0;
        final int grade = 1 + r.nextInt(100);
        final boolean current = (i < N_CURRENT_STUDENTS);
    }
}
```

```
        students[i] = new Student(firstName, lastName, age, grade, current);  
    }  
    return students;  
}
```

1. Compute the average age of all actively enrolled students and compare execute time:
 - a) Using loop
 - b) Using stream
 - c) Using parallel stream
2. Compute the number of students (using parallel stream) with conditions:
 - a) Student is not currently active.
 - b) Age > 20
 - c) Grade < 65